Lance Belen

Student ID: 36230928

Department of Data Science and Artificial Intelligence

ITO5145 - Introduction to Data Science

Assessment 1: Data Science Project

Part B: Shell Commands

Due date: 2 June 2025

Date of submission: 2 June 2025

Table of Contents

Task 1: sold_time range of the records

Code:

```
tail -n +2 property_transaction_victoria.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f4 | \
while read -r dt; do
    [[ -z "$dt" ]] && continue
    epoch=$(date -j -f "%d/%m/%Y %H:%M" "$dt" "+%s" 2>/dev/null)
    [[ -z "$epoch" ]] && continue
    echo "$epoch"
done | sort -n | head -n1 | xargs -I{} date -r {} "+%d/%m/%Y %H:%M"

tail -n +2 property_transaction_victoria.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f4 | \
while read -r dt; do
    [[ -z "$dt" ]] && continue
    epoch=$(date -j -f "%d/%m/%Y %H:%M" "$dt" "+%s" 2>/dev/null)
    [[ -z "$epoch" ]] && continue
    echo "$epoch"
done | sort -n | tail -n1 | xargs -I{} date -r {} "+%d/%m/%Y %H:%M"
```

Answer:

- The sold_time range is from 01/01/2021 01:14 to 31/12/2021 22:58

Explanation:

- The command reads the 4th column of the dataset (sold_time) and reads it through a loop for processing. It ensures that the entry is not empty, tries to convert it to epoch, and ensures that the conversion returns as expected.

- It then passes the epoch converted date time values into the next operation for sorting.
- Finally, it gets both the head (first element after sorting) and tail (last element after sorting) to find the minimum and maximum datetime and convert it back to the original format for printing.

Task 2: preprocess ID and sort_time columns

Code:

```
tail -n +2 property_transaction_victoria.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f1 | \
awk 'length($0) != 6 || !/^[0-9]+$/ { count++ } END { print count }'
```

Answer:

- There are 10 lines with an ID that is not a number or 6 digits long.

Explanation:

- The command goes through the dataset and increments a counter if the entry for the first column (ID) is not 6 characters long or if it contains a character that is not numerical.

Code:

```
head -n 1 property_transaction_victoria.csv > filtered_property.csv
tail -n +2 property_transaction_victoria.csv | \
awk -F',' 'length($1) == 6 && $1 ~ /^[0-9]+$/ {
    split($4, dt_parts, " ");
    $4 = dt_parts[1]
```

```
    OFS = ",";
    print $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14
}' >> filtered_property.csv
```

Explanation:

- We first output the header (first entry of the csv) to keep the name of the columns.
- Next, we get the second entry in the csv and onwards, and process it. If it meets the condition that the length of the value in the ID column is 6 and all of the characters are numerical, then we keep that entry and process the fourth column (sold_time) to remove the time and only keep the date.
- Finally, we append it to the target csv file - filtered_property.csv

Task 3: Investigate the description column

a) **Number of records that contain both "Alfresco" and "Renovation" in the description (case insensitive).**

Code:

```
tail -n +2 filtered_property.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f14 | \
awk '(tolower($0) ~ /alfresco/ && tolower($0) ~ /renovation/) { count++ }  END { print count }'
```

Answer:

- There are 548 records that contain both "Alfresco" and "Renovation".

Explanation:

- The command matches entries where the 14th column (description) contains both "Alfresco" and "Renovation", case insensitive with the tolower() function.

b) Number of records that contain the property size information.

Code:

```
tail -n +2 filtered_property.csv | \
iconv -c -f utf-8 -t utf-8 | \
cut -d',' -f14 | \
awk '{
  desc = tolower($0);
  if (desc ~ /[0-9]+ *((m2|m²|sqm|sq\.? *metres|square *metres|sq *m))/) {
    count++
  }
} END { print count }'
```

Answer:

- There are 37,747 records that contain the property size information that would have metrics such as m2, sqm, metres, metres square, sq. metre, sq metre, square metre.

Explanation:

- Similar to ***(a)***, The command matches entries where the 14th column (description) contains a unit of measurement defined via regex.

Task 4: Filtered dataset

Code:

```
head -n 1 filtered_property.csv | cut -d',' -f1,4,5,6,7,8,11,12,14 > filtered_dataset_task4.csv


tail -n +2 filtered_property.csv | \
iconv -c -f utf-8 -t utf-8 | \
awk -F',' '{
  split($4, sold_time, "/");
  month = sold_time[2] + 0;
  odd = (month % 2 == 1);
  townhouse = (tolower($12) ~ /townhouse/);
  area_num = $11;
  gsub(/[^0-9.]/, "", area_num);
  greater_than_300 = (area_num + 0 > 300);
  if (odd && townhouse && greater_than_300) {
    OFS = ",";
    print $1, $4, $5, $6, $7, $8, $11, $12, $14;
  }
}' >> filtered_dataset_task4.csv
```

```
echo "First sold_time:"
tail -n +2 filtered_dataset_task4.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f2 | \
while read -r dt; do
  [[ -z "$dt" ]] && continue
  epoch=$(date -j -f "%d/%m/%Y" "$dt" "+%s" 2>/dev/null)
  [[ -z "$epoch" ]] && continue
  echo "$epoch"
done | sort -n | head -n1 | xargs -I{} date -r {} "+%d/%m/%Y %H:%M"

echo "Last sold_time:"
tail -n +2 filtered_dataset_task4.csv | iconv -c -f utf-8 -t utf-8 | cut -d',' -f2 | \
while read -r dt; do
  [[ -z "$dt" ]] && continue
  epoch=$(date -j -f "%d/%m/%Y" "$dt" "+%s" 2>/dev/null)
  [[ -z "$epoch" ]] && continue
  echo "$epoch"
done | sort -n | tail -n1 | xargs -I{} date -r {} "+%d/%m/%Y %H:%M"
```

Answer:

- The first and last sold_time of the filtered dataset is 04/01/2021 and 30/11/2021 respectively.

Explanation:

- Similar to previous tasks, the command above preprocesses the dataset first and filters out the records that don't meet the requirements (odd month, townhouse, and area greater than 300). Then, the result is stored in a csv file.
- Similar to the first task, the sold_time values of the filtered dataset are then converted to epoch format, which then gets sorted to get the first and last sold_time and convert it back to the original date format for printing.